T01 — Basic Modeling Tutorial

Context, Alphabet, States, and Transitions

Mariano Cerrutti

Mununu Tutorial

What is Mununu?

Mununu is a formal verification tool for reactive systems modeled as Compositional Labeled Transition Systems (CLTS).

- Model systems as finite-state machines with labeled transitions
- Verify properties using mu-calculus and LTL
- Synthesize controllers that enforce specifications
- Compose subsystems with synchronous or asynchronous semantics

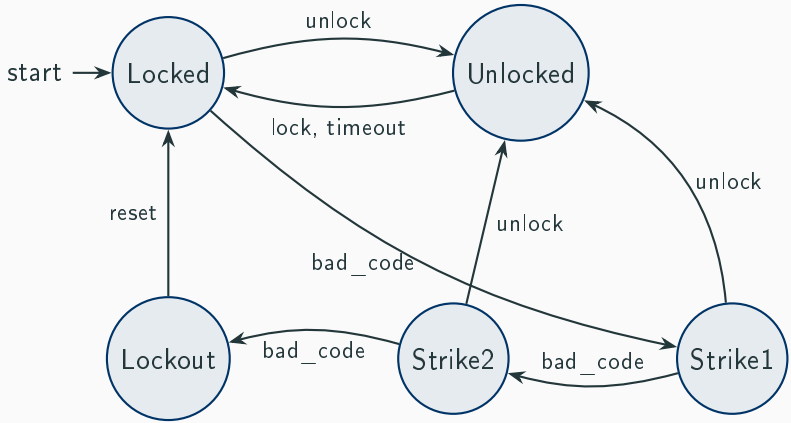
Why Model with State Machines?

Real systems are complex. State machines let us abstract away implementation details and focus on **behavior**.

Benefit	Description
Succinctness	Capture protocol logic in a handful of states
Precision	Every transition is explicit — no hidden behavior
Composability	Build large systems from small, verified components
Verifiability	Automatically check safety, liveness, reachability

The Door Lock Controller

An industrial door lock with five states: three bad codes trigger lockout.



CTXDSL: Context and Alphabet

Every `.ctxdsl` file starts with a `context` block. The `alphabet` declares every event the system can observe.

```
context door_lock {
    alphabet {
        label unlock;
        label lock;
        label bad_code;
        label timeout;
        label reset;
    }
```

- `context` — top-level container (one per file)
- `label` — declares a unique event name

CTXDSL: Automaton and States

Inside `automata`, each `automaton` defines a finite-state machine.

```
automata {
    automaton DoorLock {
        states {
            state Locked initial;
            state Unlocked;
            state Strike1;
            state Strike2;
            state Lockout;
        }
```

- `automaton Name` — names the FSM
- `state Name initial` — marks the starting state
- `state Name` — additional states

CTXDSL: Transitions (Unlock and Relock)

Transitions connect states via labeled events.

```
transitions {
    // Correct code unlocks from any locked/strike state
    transition Locked -> Unlocked on label unlock;
    transition Strike1 -> Unlocked on label unlock;
    transition Strike2 -> Unlocked on label unlock;

    // Door relocks
    transition Unlocked -> Locked on label lock;
    transition Unlocked -> Locked on label timeout;
```

- `transition A -> B on label L` — edge from A to B on event L
- Multiple transitions can share the same label (e.g., `unlock`)

CTXDSL: Transitions (Strikes and Lockout)

Bad codes advance through strikes; three strikes trigger lockout.

```
            // Bad code advances through strikes toward lockout
            transition Locked -> Strike1 on label bad_code;
            transition Strike1 -> Strike2 on label bad_code;
            transition Strike2 -> Lockout on label bad_code;

            // Admin reset from lockout
            transition Lockout -> Locked on label reset;
        }
```

The `reset` event is the only way to recover from `Lockout`.

CTXDSL: Formulas — Safety and Reachability

Properties are specified in the `mu_formulas` block.

```
mu_formulas {
    // Safety: DoorLock always stays in a valid state
    formula safety_invariant {
        over DoorLock;
        body = nu X. ([] X);
    }

    // Reachability: the Lockout state is reachable (non-trivial)
    formula lockout_reachable {
        over DoorLock;
        body = mu X. (Lockout || <> X);
    }
```

CTXDSL: Formulas — Safety and Reachability

- $\nu X.\, \Box X$ — **safety**: every reachable state is valid
- $\mu X.\, (p \vee \Diamond X)$ — **reachability**: state p can be reached

CTXDSL: Formulas — Recovery and Cycles

```
        // Recovery: from Lockout, Locked is always reachable via reset
        formula can_recover {
            over DoorLock;
            body = (! Lockout) || (mu Y. (Locked || <> Y));
        }

        // Cycle: from any state, Unlocked is reachable
        formula unlock_reachable {
            over DoorLock;
            body = mu X. (Unlocked || <> X);
        }
    }
```

- **Recovery**: from Lockout, Locked is always reachable
- **Cycle**: from any state, Unlocked is reachable

Running It

Evaluate a formula:

```
$ mununu context eval 01_basic_modeling.ctxdsl \
    --formula lockout_reachable --automaton DoorLock
Formula lockout_reachable: 5/5 states satisfying
```

Summarize the context:

```
$ mununu context summarize 01_basic_modeling.ctxdsl
Context door_lock: 1 automaton, 4 formulas
```

Syntax Summary (1/2)

Keyword	Purpose
`context`	Top-level container (one per file)
`alphabet`	Signal/event declarations
`label`	Declares one event name
`automata`	Block containing automaton definitions
`automaton`	Defines one finite-state machine
`states`	Block listing state declarations
`state`	Declares a state; `initial` marks the start

Syntax Summary (2/2)

Keyword	Purpose
`transitions`	Block listing transition rules
`transition`	One edge: `A -> B on label L`
`on`	Precedes the triggering label(s)
`mu_formulas`	Block of property specifications
`formula`	One named property
`over`	Target automaton for evaluation
`body`	The mu-calculus expression

Thank You

Mununu Tutorial

Mariano Cerrutti

vscorza@gmail.com

linkedin.com/in/mariano-cerrutti-533b1510

https://github.com/vscorza/mununu https://github.com/vscorza/mununu-ui